

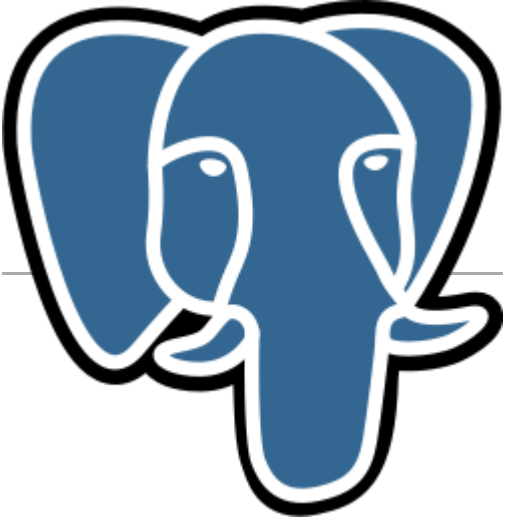
PostgreSQL

PostgreSQL (/ˈpoʊstɡrɛskjuːl/ ⓘ) ⓘ *POHST-gres-kew-EL*),^{[12][13]} also known as **Postgres**, is a free and open-source relational database management system (RDBMS) emphasizing extensibility and SQL compliance. PostgreSQL features transactions with atomicity, consistency, isolation, durability (ACID) properties, automatically updatable views, materialized views, triggers, foreign keys, and stored procedures.^[14] It is supported on all major operating systems, including Windows, Linux, macOS, FreeBSD, and OpenBSD, and handles a range of workloads from single machines to data warehouses, data lakes,^[15] or web services with many concurrent users.

The PostgreSQL Global Development Group focuses only on developing a database engine and closely related components. This core is, technically, what comprises PostgreSQL itself, but there is an extensive developer community and ecosystem that provides other important feature sets that might, traditionally, be provided by a proprietary software vendor. These include special-purpose database engine features, like those needed to support a geospatial^[16] or temporal^[17] database or features which emulate other database products.^{[18][19][20][21]} Also available from third parties are a wide variety of user and machine interface features, such as graphical user interfaces^{[22][23][24]} or load balancing and high availability toolsets.^[25] The large third-party PostgreSQL support network of people, companies, products, and projects, even though not part of The PostgreSQL Development Group, are essential to the PostgreSQL database engine's adoption and use and make up the PostgreSQL ecosystem writ large.^[26]

PostgreSQL was originally named POSTGRES, referring to its origins as a successor to the Ingres database developed at the University of California, Berkeley.^{[27][28]} In 1996, the project was renamed

PostgreSQL



The World's Most Advanced Open Source Relational Database^[1]

Other names	Postgres
Developer	PostgreSQL Global Development Group ^[2]
Release	8 July 1996 ^[3]
Stable release	18.4 ^[4]  / 14 May 2026
Preview release	19 beta 2 ^[5]  / 16 July 2026
Written in	C (and C++ for the LLVM dependency)
Type	RDBMS
License	PostgreSQL License (free and open-source, permissive) ^{[6][7][8]}
Website	www.postgresql.org (https://www.postgresql.org)
Repository	git.postgresql.org/gitweb/?p=postgresql.git (https://git.postgresql.org/gitweb/?p=postgresql.git)

PostgreSQL License

Publisher	PostgreSQL Global Development Group
------------------	-------------------------------------

"PostgreSQL" to reflect its support for SQL. After a review in 2007, the development team decided to keep the name PostgreSQL and the alias Postgres.^[29]

History

Ingres and University POSTGRES (1982–1994)

PostgreSQL evolved from the Ingres project at the University of California, Berkeley. In 1982, the leader of the Ingres team, Michael Stonebraker, left Berkeley to make a proprietary version of Ingres.^[27] He returned to Berkeley in 1985, and began a post-Ingres project to address the problems with contemporary database systems that had become increasingly clear during the early 1980s. He won the Turing Award in 2014 for these and other projects,^[30] and techniques pioneered in them.

The new project, POSTGRES, aimed to add the fewest features needed to completely support data types.^[31] These features included the ability to define types and to fully describe relationships – something used widely, but maintained entirely by the user. In POSTGRES, the database understood relationships, and could retrieve information in related tables in a natural way using *rules*. POSTGRES used many of the ideas of Ingres, but not its code.^[32]

Starting in 1986, published papers described the basis of the system, and a prototype version was shown at the 1988 ACM SIGMOD Conference. The team released version 1 to a small number of users in June 1989, followed by version 2 with a re-written rules system in June 1990. Version 3, released in 1991, again re-wrote the rules system, and added support for multiple storage managers^[33] and an improved query engine. By 1993, the number of users began to overwhelm the project with requests for support and features. After releasing version 4.2^[34] on June 30, 1994 – primarily a cleanup – the project ended. Berkeley released POSTGRES under an MIT License variant, which enabled other developers to use the code for any use. At the time, POSTGRES used an Ingres-influenced POSTQUEL query language interpreter, which could be interactively used with a console application named monitor.

Postgres95 (1994–1996)

In 1994, Berkeley graduate students Andrew Yu and Jolly Chen replaced the POSTQUEL query language interpreter with one for the SQL query language, creating Postgres95. The monitor console was also replaced by psql. Yu and Chen announced the first version (0.01) to beta testers on May 5, 1995. Version 1.0 of Postgres95 was announced on September 5, 1995, with a more liberal license that enabled the software to be freely modifiable.

On July 8, 1996, Marc Fournier at Hub.org Networking Services provided the first non-university development server for the open-source development effort.^[3] With the participation of Bruce Momjian and Vadim B. Mikheev, work began to stabilize the code inherited from Berkeley.

	Regents of the University of California
<u>Debian FSG compatible</u>	Yes ^{[9][10]}
<u>FSF approved</u>	Yes ^[11]
<u>OSI approved</u>	Yes ^[8]
<u>GPL compatible</u>	Yes
<u>Copyleft</u>	No
<u>Linking from code with a different license</u>	Yes
<u>Website</u>	postgresql.org/about/licence (http://postgresql.org/about/licence)

PostgreSQL (1996–present)

In 1996, the project was renamed to PostgreSQL to reflect its support for SQL. The online presence at the website PostgreSQL.org began on October 22, 1996.^[35] The first PostgreSQL release formed version 6.0 on January 29, 1997. Since then developers and volunteers around the world have maintained the software as The PostgreSQL Global Development Group.^[2]

The project continues to make releases available under its free and open-source software PostgreSQL License. Code comes from contributions from proprietary vendors, support companies, and open-source programmers.

As of 2025, PostgreSQL is on major release version 18 which is notable in implementing asynchronous I/O (AIO), enabling database users to perform concurrent I/O tasks like readahead and sequential scan.^[36]

Multiversion concurrency control (MVCC)

PostgreSQL manages concurrency through multiversion concurrency control (MVCC), which gives each transaction a "snapshot" of the database, allowing changes to be made without affecting other transactions. This largely eliminates the need for read locks, and ensures the database maintains ACID principles. PostgreSQL offers four levels of transaction isolation: Read Uncommitted, Read Committed, Repeatable Read and Serializable. Because PostgreSQL is immune to dirty reads, requesting a Read Uncommitted transaction isolation level provides read committed instead. PostgreSQL supports full serializability via the serializable snapshot isolation (SSI) method.^[37] The PostgreSQL MVCC implementation is prone to performance issues that require tuning when under a heavy write load which updates existing rows.^[38]

Storage and replication

Replication

PostgreSQL includes built-in binary replication based on shipping the changes (write-ahead logs (WAL)) to replica nodes asynchronously, with the ability to run read-only queries against these replicated nodes. This allows splitting read traffic among multiple nodes efficiently. Earlier replication software that allowed similar read scaling normally relied on adding replication triggers to the master, increasing load.

PostgreSQL includes built-in synchronous replication^[39] that ensures that, for each write transaction, the master waits until at least one replica node has written the data to its transaction log. Unlike other database systems, the durability of a transaction (whether it is asynchronous or synchronous) can be specified per-database, per-user, per-session or even per-transaction. This can be useful for workloads that do not require such guarantees, and may not be wanted for all data as it slows down performance due to the requirement of the confirmation of the transaction reaching the synchronous standby.

Standby servers can be synchronous or asynchronous. Synchronous standby servers can be specified in the configuration which determines which servers are candidates for synchronous replication. The first in the list that is actively streaming will be used as the current synchronous server. When this fails, the

system fails over to the next in line.

PostgreSQL's replication can trigger conflicts between the primary and standby servers, particularly whenever tuples are prematurely deleted from the standby server despite ongoing queries accessing them.^[40] To address this, PostgreSQL includes a feedback flag^[41] where the standby server proactively informs the primary server of any ongoing queries to mitigate this type of conflict.

Synchronous multi-master replication is not included in the PostgreSQL core. Postgres-XC which is based on PostgreSQL provides scalable synchronous multi-master replication.^[42] It is licensed under the same license as PostgreSQL. A related project is called Postgres-XL. Postgres-R is yet another fork.^[43] Bidirectional replication (BDR) is an asynchronous multi-master replication system for PostgreSQL.^[44]

Tools such as repmgr make managing replication clusters easier.

Several asynchronous trigger-based replication packages are available. These remain useful even after introduction of the expanded core abilities, for situations where binary replication of a full database cluster is inappropriate:

- Slony-I
- Londiste, part of SkyTools (developed by Skype)
- Bucardo multi-master replication (developed by Backcountry.com)^[45]
- SymmetricDS multi-master, multi-tier replication

Indexes

PostgreSQL includes built-in support for regular B-tree and hash table indexes, and four index access methods: generalized search trees (GiST), generalized inverted indexes (GIN), Space-Partitioned GiST (SP-GiST)^[46] and Block Range Indexes (BRIN). In addition, user-defined index methods can be created, although this is quite an involved process. Indexes in PostgreSQL also support the following features:

- Expression indexes can be created with an index of the result of an expression or function, instead of simply the value of a column.
- Partial indexes, which only index part of a table, can be created by adding a WHERE clause to the end of the CREATE INDEX statement. This allows for a smaller index to be created.
- The planner is able to use multiple indexes together to satisfy complex queries, using temporary in-memory bitmap index operations (useful for data warehouse applications for joining a large fact table to smaller dimension tables such as those arranged in a star schema).
- k-nearest neighbors (k-NN) indexing (also referred to KNN-GiST^[47]) provides efficient searching of "closest values" to that specified, useful to finding similar words, or close objects or locations with geospatial data. This is achieved without exhaustive matching of values.
- Index-only scans often allow the system to fetch data from indexes without ever having to access the main table.
- Block Range Indexes (BRIN).

Schemas

PostgreSQL schemas are namespaces, allowing objects of the same kind and name to co-exist in a single database. They are not to be confused with a database schema—the abstract, structural, organizational specification which defines how every table's data relates to data within other tables. All PostgreSQL database objects, except for a few global objects such as roles and tablespaces, exist within a schema. They cannot be nested, schemas cannot contain schemas. The permission system controls access to schemas and their content. By default, newly created databases have only a single schema called *public* but other schemas can be added and the public schema isn't mandatory.

A `search_path` setting determines the order in which PostgreSQL checks schemas for unqualified objects (those without a prefixed schema). By default, it is set to `$user, public` (`$user` refers to the currently connected database user). This default can be set on a database or role level, but as it is a session parameter, it can be freely changed (even multiple times) during a client session, affecting that session only.

Non-existent schemas, or other schemas not accessible to the logged-in user, that are listed in `search_path` are silently skipped during object lookup.

New objects are created in whichever valid schema (one that can be accessed) appears first in the `search_path`.

Data types

A wide variety of native data types are supported, including:

- Boolean
- Arbitrary-precision numerics
- Character (text, varchar, char)
- Binary
- Date/time (timestamp/time with/without time zone, date, interval)
- Money
- Enum
- Bit strings
- Text search type
- Composite
- HStore, an extension enabled key–value store within PostgreSQL^[48]
- Arrays (variable-length and can be of any data type, including text and composite types) up to 1 GB in total storage size
- Geometric primitives
- IPv4 and IPv6 addresses
- Classless Inter-Domain Routing (CIDR) blocks and MAC addresses
- XML supporting XPath queries
- Universally unique identifier (UUID)
- JavaScript Object Notation (JSON), and a faster binary JSONB (not the same as BSON^[49])

In addition, users can create their own data types which can usually be made fully indexable via PostgreSQL's indexing infrastructures – GiST, GIN, SP-GiST. Examples of these include the geographic information system (GIS) data types from the PostGIS project for PostgreSQL.

There is also a data type called a *domain*, which is the same as any other data type but with optional constraints defined by the creator of that domain. This means any data entered into a column using the domain will have to conform to whichever constraints were defined as part of the domain.

A data type that represents a range of data can be used which are called range types. These can be discrete ranges (e.g. all integer values 1 to 10) or continuous ranges (e.g., any time between 10:00 am and 11:00 am). The built-in range types available include ranges of integers, big integers, decimal numbers, time stamps (with and without time zone) and dates.

Custom range types can be created to make new types of ranges available, such as IP address ranges using the inet type as a base, or float ranges using the float data type as a base. Range types support inclusive and exclusive range boundaries using the `[ ]` and `( )` characters respectively. (e.g., `[4,9)` represents all integers starting from and including 4 up to but not including 9.) Range types are also compatible with existing operators used to check for overlap, containment, right of etc.

User-defined objects

New types of almost all objects inside the database can be created, including:

- Casts
- Conversions
- Data types
- Data domains
- Functions, including aggregate functions and window functions
- Indexes including custom indexes for custom types
- Operators (existing ones can be overloaded)
- Procedural languages

Inheritance

Tables can be set to inherit their characteristics from a *parent* table. Data in child tables will appear to exist in the parent tables, unless data is selected from the parent table using the ONLY keyword, i.e. **SELECT * FROM ONLY parent_table;** Adding a column in the parent table will cause that column to appear in the child table.

Inheritance can be used to implement table partitioning, using either triggers or rules to direct inserts to the parent table into the proper child tables.

This feature is not fully supported. In particular, table constraints are not currently inheritable. All check constraints and not-null constraints on a parent table are automatically inherited by its children. Other types of constraints (unique, primary key, and foreign key constraints) are not inherited.

Inheritance provides a way to map the features of generalization hierarchies depicted in entity–relationship diagrams (ERDs) directly into the PostgreSQL database.

Other storage features

- Referential integrity constraints including foreign key constraints, column constraints, and row checks
- Binary and textual large-object storage
- Tablespaces
- Per-column collation
- Online backup
- Point-in-time recovery, implemented using write-ahead logging
- In-place upgrades with `pg_upgrade` for less downtime

Control and connectivity

Foreign data wrappers

PostgreSQL can link to other systems to retrieve data via foreign data wrappers (FDWs).^[50] These can take the form of any data source, such as a file system, another relational database management system (RDBMS), or a web service. This means that regular database queries can use these data sources like regular tables, and even join multiple data-sources together.

Interfaces

PostgreSQL supports a binary communication protocol that allows applications to connect to the database server. The protocol is versioned (currently 3.0, as of PostgreSQL 7.4) and has a detailed specification.^[51]

The official client implementation of this communication protocol is a C API, `libpq`.^[52] In addition, the officially supported ECPG tool allows SQL commands to be embedded in C code.^[53] Both are part of the standard PostgreSQL distribution.^[54]

Third-party libraries for connecting to PostgreSQL are available for many programming languages, including C++,^[55] Java,^[56] Julia,^{[57][58][59]} Python,^[60] Node.js,^[61] Go,^[62] and Rust.^[63]

Procedural languages

Procedural languages allow developers to extend the database with custom subroutines (functions), often called stored procedures. These functions can be used to build database triggers (functions invoked on modification of certain data) and custom data types and aggregate functions.^[64] Procedural languages can also be invoked without defining a function, using a `DO` command at SQL level.^[65]

Languages are divided into two groups: Procedures written in *safe* languages are sandboxed and can be safely created and used by any user. Procedures written in *unsafe* languages can only be created by superusers, because they allow bypassing a database's security restrictions, but can also access sources external to the database. Some languages like Perl provide both safe and unsafe versions.

PostgreSQL has built-in support for three procedural languages:

- Plain SQL (safe). Simpler SQL functions can get expanded inline into the calling (SQL) query, which saves function call overhead and allows the query optimizer to "see inside" the

function.

- Procedural Language/PostgreSQL (PL/pgSQL) (safe), which resembles Oracle's Procedural Language for SQL (PL/SQL) procedural language and SQL/Persistent Stored Modules (SQL/PSM).
- C (unsafe), which allows loading one or more custom shared library into the database. Functions written in C offer the best performance, but bugs in code can crash and potentially corrupt the database. Most built-in functions are written in C.

In addition, PostgreSQL allows procedural languages to be loaded into the database through extensions. Three language extensions are included with PostgreSQL to support Perl, Tcl, and Python. For Python, the current Python 3 is used, and the discontinued Python 2 is no longer supported as of PostgreSQL 15. Both were supported previously, defaulting to Python 2, while old and new versions couldn't be used in the same session.^[66] External projects provide support for many other languages,^[67] including PL/Java, JavaScript (PL/V8), PL/Julia,^[59] PL/R,^[68] PL/Ruby, and others.

Triggers

Triggers are events triggered by the action of SQL data manipulation language (DML) statements. For example, an INSERT statement might activate a trigger that checks if the values of the statement are valid. Most triggers are only activated by either INSERT or UPDATE statements.

Triggers are fully supported and can be attached to tables. Triggers can be per-column and conditional, in that UPDATE triggers can target specific columns of a table, and triggers can be told to execute under a set of conditions as specified in the trigger's WHERE clause. Triggers can be attached to views by using the INSTEAD OF condition. Multiple triggers are fired in alphabetical order. In addition to calling functions written in the native PL/pgSQL, triggers can also invoke functions written in other languages like PL/Python or PL/Perl.

Asynchronous notifications

PostgreSQL provides an asynchronous messaging system that is accessed through the NOTIFY, LISTEN and UNLISTEN commands. A session can issue a NOTIFY command, along with the user-specified channel and an optional payload, to mark a particular event occurring. Other sessions are able to detect these events by issuing a LISTEN command, which can listen to a particular channel. This functionality can be used for a wide variety of purposes, such as letting other sessions know when a table has updated or for separate applications to detect when a particular action has been performed. Such a system prevents the need for continuous polling by applications to see if anything has yet changed, and reducing unnecessary overhead. Notifications are fully transactional, in that messages are not sent until the transaction they were sent from is committed. This eliminates the problem of messages being sent for an action being performed which is then rolled back.

Many connectors for PostgreSQL provide support for this notification system (including libpq, JDBC, Npgsql, psycopg and node.js) so it can be used by external applications.

PostgreSQL can act as an effective, persistent "pub/sub" server or job server by combining LISTEN with FOR UPDATE SKIP LOCKED.^{[69][70][71]}

Rules

Rules allow the "query tree" of an incoming query to be rewritten; they are an, automatically invoked, macro language for SQL. "Query Re-Write Rules" are attached to a table/class and "Re-Write" the incoming DML (select, insert, update, and/or delete) into one or more queries that either replace the original DML statement or execute in addition to it. Query Re-Write occurs after DML statement parsing and before query planning.

The functionality rules provide was, in almost every way, later duplicated with the introduction of newer types of triggers. The use of triggers is usually preferred over rules as it is easier to reason about trigger behavior and interactions than when equivalent rules are used.

Other querying features

- Transactions
- Full-text search
- Views
 - Materialized views^[72]
 - Updateable views^[73]
 - Recursive views^[74]
- Inner, outer (full, left, and right), and cross joins
- Sub-selects
 - Correlated sub-queries^[75]
- Regular expressions^[76]
- Common table expressions and writable common table expressions
- Encrypted connections via Transport Layer Security (TLS); current versions do not use vulnerable SSL, even with that configuration option^[77]
- Domains
- Savepoints
- Two-phase commit
- The Oversized-Attribute Storage Technique (TOAST) is used to transparently store large table attributes (such as big MIME attachments or XML messages) in a separate area, with automatic compression.
- Embedded SQL is implemented using preprocessor. SQL code is first written embedded into C code. Then code is run through ECPG preprocessor, which replaces SQL with calls to code library. Then code can be compiled using a C compiler. Embedding works also with C++ but it does not recognize all C++ constructs.

Concurrency model

PostgreSQL server is process-based (not threaded), and uses one operating system process per database session. Multiple sessions are automatically spread across all available CPUs by the operating system. Many types of queries can also be parallelized across multiple background worker processes, taking advantage of multiple CPUs or cores.^[78] Client applications can use threads and create multiple database connections from each thread.^[79]

Security

PostgreSQL manages its internal security on a per-role basis. A role is generally regarded to be a user (a role that can log in), or a group (a role of which other roles are members). Permissions can be granted or revoked on any object down to the column level, and can allow or prevent the visibility/creation/alteration/deletion of objects at the database, schema, table, and row levels.

PostgreSQL's SECURITY LABEL feature (extension to SQL standards), allows for additional security; with a bundled loadable module that supports label-based mandatory access control (MAC) based on Security-Enhanced Linux (SELinux) security policy.^{[80][81]}

PostgreSQL natively supports a broad number of external authentication mechanisms, including:

- Password: either SCRAM-SHA-256,^[82] MD5 or plain-text
- Generic Security Services Application Program Interface (GSSAPI)
- Security Support Provider Interface (SSPI)
- Kerberos
- ident (maps O/S user-name as provided by an ident server to database user-name)
- Peer (maps local user name to database user name)
- Lightweight Directory Access Protocol (LDAP)
 - Active Directory (AD)
- RADIUS
- Certificate
- Pluggable authentication module (PAM)

The GSSAPI, SSPI, Kerberos, peer, ident and certificate methods can also use a specified "map" file that lists which users matched by that authentication system are allowed to connect as a specific database user.

These methods are specified in the cluster's host-based authentication configuration file (`pg_hba.conf`), which determines what connections are allowed. This allows control over which user can connect to which database, where they can connect from (IP address, IP address range, domain socket), which authentication system will be enforced, and whether the connection must use Transport Layer Security (TLS).

Standards compliance

PostgreSQL claims high, but not complete, conformance with the latest SQL standard ("as of the version 17 release in September 2024, PostgreSQL conforms to at least 170 of the 177 mandatory features for SQL:2023 Core conformance", and no other databases fully conformed to it^[83]). One exception is the handling of unquoted identifiers like table or column names. In PostgreSQL they are folded, internally, to lower case characters^[84] whereas the standard says that unquoted identifiers should be folded to upper case. Thus, FOO should be equivalent to FOO not foo according to the standard. Other shortcomings concern the absence of temporal tables allowing automatic logging of row versions during transactions with the possibility of browsing in time (FOR SYSTEM TIME predicate), although relatively SQL compliant third-party extensions are available.^[17]

Benchmarks and performance

Many informal performance studies of PostgreSQL have been done.^[85] Performance improvements aimed at improving scalability began heavily with version 8.1. Simple benchmarks between version 8.0 and version 8.4 showed that the latter was more than ten times faster on read-only workloads and at least 7.5 times faster on both read and write workloads.^[86]

The first industry-standard and peer-validated benchmark was completed in June 2007, using the Sun Java System Application Server (proprietary version of GlassFish) 9.0 Platform Edition, UltraSPARC T1-based Sun Fire server and PostgreSQL 8.2.^[87] This result of 778.14 SPECjAppServer2004 JOPS@Standard compares favourably with the 874 JOPS@Standard with Oracle 10 on an Itanium-based HP-UX system.^[85]

In August 2007, Sun submitted an improved benchmark score of 813.73 SPECjAppServer2004 JOPS@Standard. With the system under test at a reduced price, the price/performance improved from \$84.98/JOPS to \$70.57/JOPS.^[88]

The default configuration of PostgreSQL uses only a small amount of dedicated memory for performance-critical purposes such as caching database blocks and sorting. This limitation is primarily because older operating systems required kernel changes to allow allocating large blocks of shared memory.^[89] PostgreSQL.org provides advice on basic recommended performance practice in a wiki.^[90]

In April 2012, Robert Haas of EnterpriseDB demonstrated PostgreSQL 9.2's linear CPU scalability using a server with 64 cores.^[91]

Matloob Khushi performed benchmarking between PostgreSQL 9.0 and MySQL 5.6.15 for their ability to process genomic data. In his performance analysis he found that PostgreSQL extracts overlapping genomic regions eight times faster than MySQL using two datasets of 80,000 each forming random human DNA regions. Insertion and data uploads in PostgreSQL were also better, although general searching ability of both databases was almost equivalent.^[92]

Platforms

PostgreSQL is available for the following operating systems: Linux (all recent distributions), 64-bit ARM and x86-64 installers available and tested for macOS version 10.14 and newer,^[93] Windows (with installers available and tested for 64-bit Windows Server 2022 and 2016^[94]), FreeBSD, OpenBSD,^[95] NetBSD, DragonFlyBSD, and these without official (though unofficial likely available) binary executables, Solaris,^[96] and illumos.

PostgreSQL can be expected to work on any of the following instruction set architectures (and operating systems): 64-bit x86-64 and 32-bit x86 on Windows and other operating systems; these are supported on other than Windows: 64-bit ARM^[97] and the older 32-bit ARM, including older such as ARMv6 in Raspberry Pi^[98], RISC-V, z/Architecture, S/390, PowerPC (incl. 64-bit Power ISA), SPARC (also 64-bit), MIPS and PA-RISC. It was also known to work on some other platforms (while not been tested on for years, i.e. for latest versions).^[99]

Database administration

Open source front-ends and tools for administering PostgreSQL include:

psql

The primary front-end for PostgreSQL is the `psql` command-line program, which can be used to enter SQL queries directly, or execute them from a file. In addition, psql provides a number of meta-commands and various shell-like features to facilitate writing scripts and automating a wide variety of tasks; for example tab completion of object names and SQL syntax.

pgAdmin

The pgAdmin package is a free and open-source graphical user interface (GUI) administration tool for PostgreSQL, which is supported on many computer platforms.^[101] The program is available in more than a dozen languages. The first prototype, named pgManager, was written for PostgreSQL 6.3.2 from 1998, and rewritten and released as pgAdmin under the GNU General Public License (GPL) in later months. The second incarnation (named pgAdmin II) was a complete rewrite, first released on January 16, 2002. The third version, pgAdmin III, was originally released under the Artistic License and then released under the same license as PostgreSQL. Unlike prior versions that were written in Visual Basic, pgAdmin III is written in C++, using the wxWidgets^[102] framework allowing it to run on most common operating systems. The query tool includes a scripting language called pgScript for supporting admin and development tasks. In December 2014, Dave Page, the pgAdmin project founder and primary developer,^[103] announced that with the shift towards web-based models, work has begun on pgAdmin 4 with the aim to facilitate cloud deployments.^[104] In 2016, pgAdmin 4 was released. The pgAdmin 4 backend was written in Python, using Flask and the Qt framework.^[105]

phpPgAdmin

phpPgAdmin is a web-based administration tool for PostgreSQL written in PHP and based on the popular phpMyAdmin interface originally written for MySQL administration.^[106]

PostgreSQL Studio

PostgreSQL Studio allows users to perform essential PostgreSQL database development tasks from a web-based console. PostgreSQL Studio allows users to work with cloud databases without the need to open firewalls.^[107]

TeamPostgreSQL

AJAX/JavaScript-driven web interface for PostgreSQL. Allows browsing, maintaining and creating data and database objects via a web browser. The interface offers tabbed SQL editor with autocompletion, row editing widgets, click-through foreign key navigation between rows and tables, *favorites* management for commonly used scripts, among other features. Supports SSH for both the web interface and the database connections. Installers are available for Windows, Macintosh, and Linux, and a simple cross-platform archive that runs from a script.^[108]

LibreOffice, OpenOffice.org

LibreOffice and OpenOffice.org Base can be used as a front-end for PostgreSQL.^{[109][110]}

pgBadger

The pgBadger PostgreSQL log analyzer generates detailed reports from a PostgreSQL log file.^[111]

pgDevOps

pgDevOps is a suite of web tools to install & manage multiple PostgreSQL versions, extensions, and community components, develop SQL queries, monitor running databases and find performance problems.^[112]

Adminer

psql Session Example^[100]

```
regression=# select foo;
ERROR:  column "foo" does not exist
CONTEXT:  PL/pgSQL function "test1" while casting return value to function's return type
LINE 1: select foo;
        ^
regression=# \q
```

```
peter@localhost testdb=> \a \t \x
Output format is aligned.
Tuples only is off.
Expanded display is on.
```

```
regression=# select '\x';
WARNING:  nonstandard use of escape in a string literal
LINE 1: select '\x';
        ^
HINT:  Use the escape string syntax for escapes, e.g., E'\r\n'.
?column?
-----
 x
(1 row)

regression=# select E'\x';
```

```
piro=> \set foo 30;
piro=> select * from test where foo <= :foo;
foo | bar
-----+-----
 10 |
 20 |
(2 rows)
```

```
testdb=> \set foo 'my_table'
testdb=> SELECT * FROM :foo";

testdb=> \set content `cat my_file.txt`
testdb=> INSERT INTO my_table VALUES (:content');
```

```
regression=# select (
regression(# 1);
?column?
-----
 1
(1 row)
```

```
piro=> select (
piro(> '
piro'> ' || $$
piro$> $$)
piro-> from "
piro"> foo";
ERROR:  relation "
foo" does not exist
LINE 5: from "
        ^
```

```
testdb=> CREATE TABLE my_table (
first integer not null default 0,
```

```
second text) ; -- end of command
CREATE TABLE

=# SELECT '0x10'::mpz AS "hex", '10'::mpz AS "dec",
-#      '010'::mpz AS oct, '0b10'::mpz AS bin; -- Table output
 hex | dec | oct | bin
-----+-----+-----+-----
  16 |  10 |   8 |   2
(1 row)
```

```
regression=# select schemaname from pg_tables limit 3; -- One field output
schemaname
-----
pg_catalog
pg_catalog
pg_catalog
(3 rows)

=# select 10.0, 1e-6, 1E+6;
?column? | ?column? | ?column?
-----+-----+-----
   10.0 | 0.000001 | 1000000
(1 row)

regression=# begin;
BEGIN
regression=# create table asdf (foo serial primary key);
NOTICE: CREATE TABLE will create implicit sequence "asdf_foo_seq" for serial column "asdf.foo"
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "asdf_pkey" for table "asdf"
CREATE TABLE
regression=# insert into asdf values (10) returning foo;
foo
-----
   10
(1 row)

INSERT 0 1
regression=# ROLLBACK ;
ROLLBACK
```

Adminer is a simple web-based administration tool for PostgreSQL and others, written in PHP.

pgBackRest

pgBackRest is a backup and restore tool for PostgreSQL that provides support for full, differential, and incremental backups.^[113]

pgaudit

pgaudit is a PostgreSQL extension that provides detailed session and/or object audit logging via the standard logging facility provided by PostgreSQL.^[114]

WAL-E

WAL-E is a backup and restore tool for PostgreSQL that provides support for physical (WAL-based) backups, written in Python.^[115]

DBeaver

DBeaver is a free and open source GUI administration tool for PostgreSQL, it has Visual Entity Diagrams and Intellisense features. It also has a commercial PRO license.

Postgresus

Postgresus is an open source backup tool with GUI for scheduled backups with support of external sources (S3, NAS, FTP, Google Drive, Google Cloud, etc.) and notifications to external services (Slack, Discord, Telegram, SMTP, etc.).^[116]

A number of companies offer proprietary tools for PostgreSQL. They often consist of a universal core that is adapted for various specific database products. These tools mostly share the administration features with the open source tools but offer improvements in data modeling, importing, exporting or

reporting.

Notable users

Notable organizations and products that use PostgreSQL as the primary database include:

- Microsoft, used for a petabyte-scale “Release Quality View” (RQV) analytics dashboard, which tracks quality of Windows updates analyzing 20K types of metrics from over 800M Windows devices.^[117]
- In 2009, the social-networking website Myspace used Aster Data Systems's nCluster database for data warehousing, which was built on unmodified PostgreSQL.^{[118][119]}
- Geni.com uses PostgreSQL for their main genealogy database.^[120]
- OpenStreetMap, a collaborative project to create a free editable map of the world.^[121]
- Afilias, domain registries for .org, .info and others.^{[122][123]}
- Sony Online multiplayer online games.^[124]
- BASF, shopping platform for their agribusiness portal.^[125]
- Reddit social news website.^[126]
- Skype VoIP application, central business databases.^[127]
- Sun xVM, Sun's virtualization and datacenter automation suite.^[128]
- MusicBrainz, open online music encyclopedia.^[129]
- The International Space Station – to collect telemetry data in orbit and replicate it to the ground.^[130]
- MyYearbook social-networking site.^[131]
- Instagram, a mobile photo-sharing service.^[132]
- Disqus, an online discussion and commenting service.^[133]
- TripAdvisor, travel-information website of mostly user-generated content.^[134]
- Yandex, a Russian internet company switched its Yandex.Mail service from Oracle to Postgres.^[135]
- Amazon Redshift, part of AWS, a columnar online analytical processing (OLAP) system based on ParAccel's Postgres modifications.
- National Oceanic and Atmospheric Administration's (NOAA) National Weather Service (NWS), Interactive Forecast Preparation System (IFPS), a system that integrates data from the NEXRAD weather radars, surface, and hydrology systems to build detailed localized forecast models.^{[123][136]}
- United Kingdom's national weather service, Met Office, has begun swapping Oracle for PostgreSQL in a strategy to deploy more open source technology.^{[136][137]}
- WhitePages.com had been using Oracle and MySQL, but when it came to moving its core directories in-house, it turned to PostgreSQL. Because WhitePages.com needs to combine large sets of data from multiple sources, PostgreSQL's ability to load and index data at high rates was a key to its decision to use PostgreSQL.^[123]
- FlightAware, a flight tracking website.^[138]
- Grofers, an online grocery delivery service.^[139]
- *The Guardian* migrated from MongoDB to PostgreSQL in 2018.^[140]
- YugabyteDB implements the PostgreSQL query layer as its default SQL mode

- [OpenAI](#) uses PostgreSQL as part of its primary API service - including for ChatGPT.^{[141][142]}

Service implementations

Some notable vendors offer PostgreSQL as software as a service:

- [Heroku](#), a platform as a service provider, has supported PostgreSQL since the start in 2007.^[143] They offer value-add features like full database *roll-back* (ability to restore a database from any specified time),^[144] which is based on WAL-E, open-source software developed by Heroku.^[145]
- In January 2012, [EnterpriseDB](#) released a cloud version of both PostgreSQL and their own proprietary Postgres Plus Advanced Server with automated provisioning for failover, replication, load-balancing, and scaling. It runs on [Amazon Web Services](#).^[146] Since 2015, Postgres Advanced Server has been offered as ApsaraDB for PPAS, a relational database as a service on Alibaba Cloud.^[147]
- [VMware](#) has offered vFabric Postgres (also termed vPostgres^[148]) for private clouds on [VMware vSphere](#) since May 2012.^[149] The company announced End of Availability (EOA) of the product in 2014.^[150]
- In November 2013, [Amazon Web Services](#) announced the addition of PostgreSQL to their [Relational Database Service](#) offering.^{[151][152]}
- In November 2016, [Amazon Web Services](#) announced the addition of PostgreSQL compatibility to their cloud-native [Amazon Aurora](#) managed database offering.^[153]
- In May 2017, [Microsoft Azure](#) announced Azure Databases for PostgreSQL.^[154]
- In May 2019, [Alibaba Cloud](#) announced PolarDB for PostgreSQL.^[155]
- [Jelastic Multicloud Platform as a Service](#) has provided container-based PostgreSQL support since 2011. It also offers automated asynchronous master-slave replication of PostgreSQL.^[156]
- In June 2019, [IBM Cloud](#) announced IBM Cloud Hyper Protect DBaaS for PostgreSQL.^[157]
- In September 2020, [Crunchy Data](#) announced Crunchy Bridge.^[158]
- In October 2021, [Render](#) has provided support for managed PostgreSQL databases since 2021.^[159]
- In June 2022, [Neon.tech](#) announced Neon Serverless Postgres.^[160]
- In December 2022, [Google Cloud Platform](#) announced general availability of AlloyDB as fully managed PostgreSQL cloud service.^[161]
- In October 2023, [Nile](#) announced Nile Postgres Platform.^[162]
- In April 2024, [Google Cloud Platform](#) announced general availability of AlloyDB Omni, a downloadable version of AlloyDB designed to run on any infrastructure, including on-premises, other clouds, or edge environments.^[163]
- In September 2025, [PlanetScale.com](#) announced general availability of PlanetScale for Postgres, a managed, highly available version of Postgres.^[164]

Release history

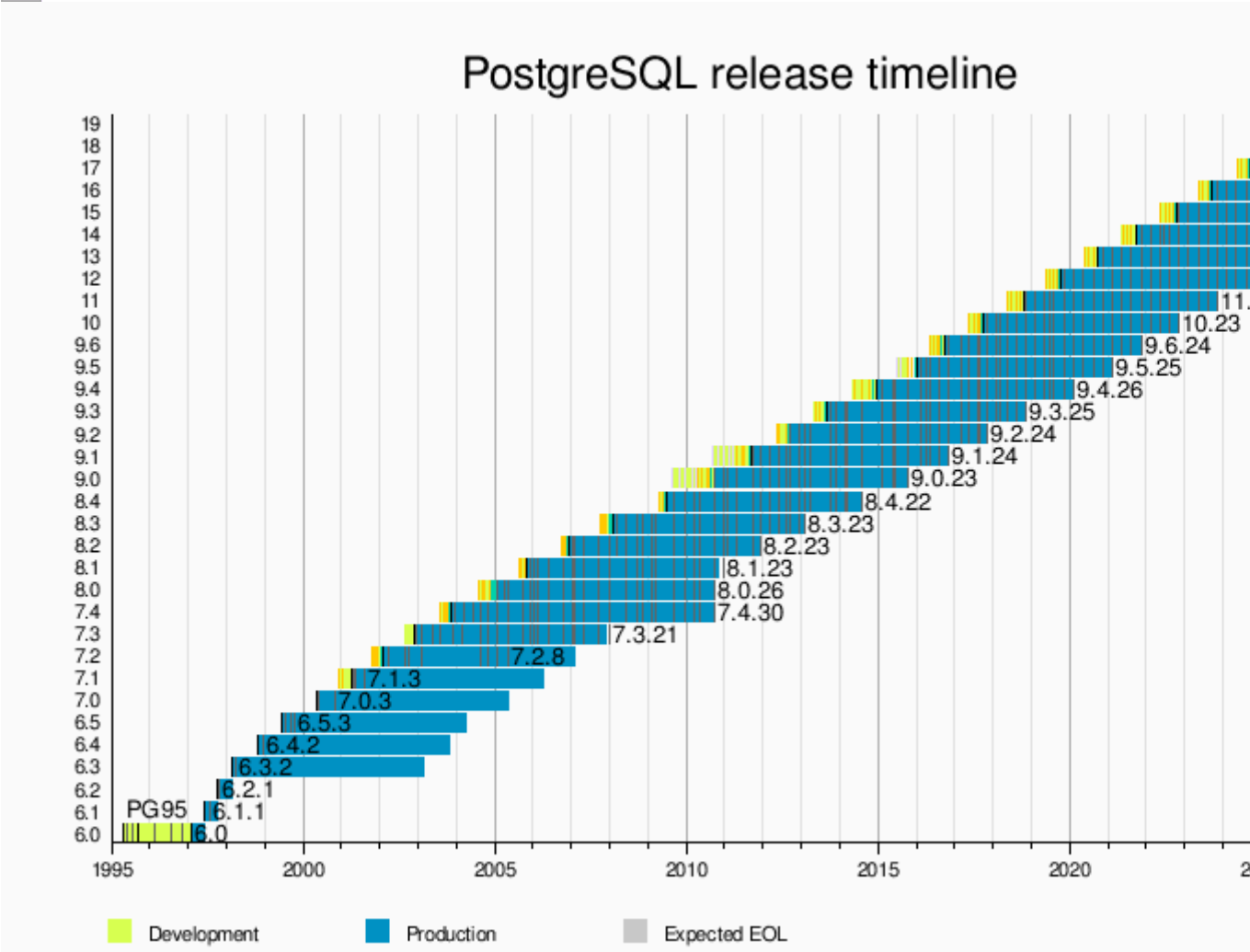
Release history

Release	First release	Latest minor version	Latest release	End of life ^[165]	Milestones
6.0	1997-01-29	—	—	—	First formal release of PostgreSQL, unique indexes, pg_dumpall utility, ident authentication
6.1	1997-06-08	6.1.1	1997-07-22	—	Multicolumn indexes, sequences, money data type, GEQO (GEnetic Query Optimizer)
6.2	1997-10-02	6.2.1	1997-10-17	—	JDBC interface, triggers, server programming interface, constraints
6.3	1998-03-01	6.3.2	1998-04-07	2003-03-01	SQL-92 subselect ability, PL/pgTCL
6.4	1998-10-30	6.4.2	1998-12-20	2003-10-30	VIEWS (then only read-only) and RULEs, PL/pgSQL
6.5	1999-06-09	6.5.3	1999-10-13	2004-06-09	MVCC, temporary tables, more SQL statement support (CASE, INTERSECT, and EXCEPT)
7.0	2000-05-08	7.0.3	2000-11-11	2004-05-08	Foreign keys, SQL-92 syntax for joins
7.1	2001-04-13	7.1.3	2001-08-15	2006-04-13	Write-ahead log, outer joins
7.2	2002-02-04	7.2.8	2005-05-09	2007-02-04	PL/Python, <u>OIDs</u> no longer required, <u>internationalization</u> of messages
7.3	2002-11-27	7.3.21	2008-01-07	2007-11-27	Schema, table function, <u>prepared query</u> ^[166]
7.4	2003-11-17	7.4.30	2010-10-04	2010-10-01	Optimization on JOINS and <u>data warehouse functions</u> ^[167]
8.0	2005-01-19	8.0.26	2010-10-04	2010-10-01	Native server on Microsoft Windows, <u>savepoints</u> , <u>tablespaces</u> , <u>point-in-time recovery</u> ^[168]
8.1	2005-11-08	8.1.23	2010-12-16	2010-11-08	Performance optimization, two-phase commit, table <u>partitioning</u> , index bitmap scan, shared row locking, roles
8.2	2006-12-05	8.2.23	2011-12-05	2011-12-05	Performance optimization, online index builds, advisory locks, warm standby ^[169]
8.3	2008-02-04	8.3.23	2013-02-07	2013-02-07	Heap-only tuples, full text search, ^[170] SQL/XML, ENUM types, <u>UUID</u> types
8.4	2009-07-01	8.4.22	2014-07-24	2014-07-24	Window functions, column-level permissions, parallel database restore, per-database collation, common table <u>expressions</u> and recursive queries ^[171]
9.0	2010-09-20	9.0.23	2015-10-08	2015-10-08	Built-in binary streaming <u>replication</u> , <u>hot standby</u> , in-place upgrade ability, 64-bit Windows ^[172]
9.1	2011-09-12	9.1.24	2016-10-27	2016-10-27	Synchronous replication, per-column <u>collations</u> , <u>unlogged tables</u> , <u>serializable</u>

					snapshot isolation, writeable common table expressions, SELinux integration, extensions, foreign tables ^[173]
9.2	2012-09-10 ^[174]	9.2.24	2017-11-09	2017-11-09	Cascading streaming replication, index-only scans, native JSON support, improved lock management, range types, <code>pg_receivexlog</code> tool, space-partitioned GiST indexes
9.3	2013-09-09	9.3.25	2018-11-08	2018-11-08	Custom background workers, data checksums, dedicated JSON operators, LATERAL JOIN, faster <code>pg_dump</code> , new <code>pg_isready</code> server monitoring tool, trigger features, view features, writeable foreign tables, materialized views , replication improvements
9.4	2014-12-18	9.4.26	2020-02-13	2020-02-13	JSONB data type, ALTER SYSTEM statement for changing config values, ability to refresh materialized views without blocking reads, dynamic registration/start/stop of background worker processes, Logical Decoding API, GiN index improvements, Linux huge page support, database cache reloading via <code>pg_prewarm</code> , reintroducing Hstore as the column type of choice for document-style data. ^[175]
9.5	2016-01-07	9.5.25	2021-02-11	2021-02-11	UPSERT, row level security, TABLESAMPLE, CUBE/ROLLUP, GROUPING SETS, and new BRIN index ^[176]
9.6	2016-09-29	9.6.24	2021-11-11	2021-11-11	Parallel query support, PostgreSQL foreign data wrapper (FDW) improvements with sort/join pushdown, multiple synchronous standbys, faster vacuuming of large table
10	2017-10-05	10.23	2022-11-10	2022-11-10	Logical replication, ^[177] declarative table partitioning, improved query parallelism
11	2018-10-18	11.22	2023-11-09	2023-11-09	Increased robustness and performance for partitioning, transactions supported in stored procedures, enhanced abilities for query parallelism, just-in-time (JIT) compiling for expressions ^{[178][179]}
12	2019-10-03	12.22	2024-11-21	2024-11-21	Improvements to query performance and space utilization; SQL/JSON path expression support; generated columns; improvements to internationalization, and authentication; new pluggable table storage interface. ^[180]
13	2020-09-24	13.23	2025-11-13	2025-11-13	Space savings and performance gains from de-duplication of B-tree index entries, improved performance for queries that use aggregates or partitioned tables, better query planning when using extended statistics, parallelized vacuuming of indexes, incremental sorting ^{[181][182]}
14	2021-09-30	14.23	2026-05-14	2026-11-12	Added SQL-standard SEARCH and CYCLE clauses for common table

					expressions, allow DISTINCT to be added to GROUP BY ^{[183][184]}
15	2022-10-13	15.18	2026-05-14	2027-11-11	Implements SQL-standard MERGE statement. PL/Python now only supports current Python 3, and plpythonu now means Python 3, no longer the discontinued Python 2.
16	2023-09-14	16.14	2026-05-14	2028-11-09	Improvements to logical replication, pg_stat_io view (for I/O metrics) ^[185]
17	2024-09-26	17.10	2026-05-14	2029-11-08	Performance boosts to the vacuum process, I/O layer, and query execution, expanding JSON functionality, more features to MERGE and improving COPY; enhances logical replication for high availability and upgrades, improvements to security, operations, monitoring, and analysis. ^[186]
18	2025-09-25	18.4	2026-05-14	2030-11-14	New I/O subsystem and asynchronous I/O enhancements. ^[187]
19	—	19 Beta 2	2026-07-16	—	

Legend: Unsupported Supported Latest version Preview version Future version

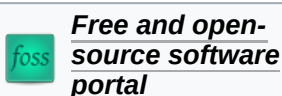


Ecosystem and Derivatives

Due to its permissive open-source license and extensible architecture, a broad ecosystem has developed around PostgreSQL. This includes numerous companies offering dedicated support and hosting, as well as several forks and derivative databases that adapt PostgreSQL for specific workloads. Notable derivatives include:

- Greenplum Database: A massively parallel processing (MPP) data warehouse based on an older version of PostgreSQL, designed for large-scale analytics.^[188]
- TimescaleDB: A time-series database delivered as a PostgreSQL extension, optimized for handling fast ingest and complex queries of time-series data.^[189]
- Amazon Aurora: A cloud-native relational database offered by Amazon Web Services that provides a PostgreSQL-compatible edition.^[190]
- Neon: An open-source, serverless implementation of PostgreSQL that separates storage and compute to offer modern development features like database branching.^[191]
- AlloyDB: A fully managed PostgreSQL-compatible Google Cloud database that separates compute and storage, designed for hybrid workloads and integrated AI capabilities.^[192]
- PlanetScale Postgres: Fully managed, highly available Postgres instances that support database branching and Database Traffic Control™.^{[164][193]}

See also



- Comparison of relational database management systems
- Database scalability
- List of databases using MVCC
- LLVM (llvmljit is the JIT engine used by PostgreSQL)

References

1. "PostgreSQL" (<https://www.postgresql.org>). Retrieved September 21, 2019. "PostgreSQL: The World's Most Advanced Open Source Relational Database"
2. "Contributor Profiles" (<https://www.postgresql.org/community/contributors/>). PostgreSQL Global Development Group. Retrieved March 14, 2017.
3. "Happy Birthday, PostgreSQL!" (<https://www.postgresql.org/about/news/978/>). PostgreSQL Global Development Group. July 8, 2008.
4. "PostgreSQL 18.4, 17.10, 16.14, 15.18, and 14.23 Released!" (<https://www.postgresql.org/about/news/postgresql-184-1710-1614-1518-and-1423-released-3297/>). May 14, 2026.
5. "PostgreSQL 19 Beta 2 Released!" (<https://www.postgresql.org/about/news/postgresql-19-beta-2-released-3350/>). July 16, 2026.
6. "License" (<https://www.postgresql.org/about/licence>). PostgreSQL Global Development Group. Retrieved September 20, 2010.

7. "PostgreSQL licence approved by OSI" (<https://web.archive.org/web/20160808093031/http://www.crynwr.com/cgi-bin/ezmlm-cgi?17:mmp:969>). Crynwr. February 18, 2010. Archived from the original (<http://www.crynwr.com/cgi-bin/ezmlm-cgi?17:mmp:969>) on August 8, 2016. Retrieved February 18, 2010.
8. "OSI PostgreSQL Licence" (<https://www.opensource.org/licenses/postgresql>). Open Source Initiative. February 20, 2010. Retrieved February 20, 2010.
9. "Debian -- Details of package postgresql in sid" (<https://packages.debian.org/unstable/postgresql>). *packages.debian.org*. Retrieved January 25, 2021.
10. "Licensing:Main" (<https://fedoraproject.org/wiki/Licensing:Main?rd=Licensing>). *FedoraProject*.
11. "PostgreSQL" (<https://directory.fsf.org/wiki/PostgreSQL>). *fsf.org*.
12. "FAQ: What is PostgreSQL? How is it pronounced? What is Postgres?" (https://wiki.postgresql.org/wiki/FAQ#What_is_PostgreSQL.3F_How_is_it_pronounced.3F_What_is_Postgres.3F). *PostgreSQL Wiki*. PostgreSQL community. Retrieved October 2, 2021.
13. "Audio sample, 5.6k MP3" (<https://www.postgresql.org/files/postgresql.mp3>).
14. "What is PostgreSQL?" (<https://www.postgresql.org/docs/current/static/intro-what-is.html>). *PostgreSQL 9.3.0 Documentation*. PostgreSQL Global Development Group. Retrieved September 20, 2013.
15. "Parquet and Postgres in the Data Lake | Crunchy Data Blog" (<https://www.crunchydata.com/blog/parquet-and-postgres-in-the-data-lake>). *Crunchy Data*. May 3, 2022. Retrieved September 19, 2024.
16. "PostGIS" (<https://postgis.net/>). *postgis.net*. December 18, 2023. Retrieved December 18, 2023. "PostGIS extends the capabilities of the PostgreSQL relational database by adding support for storing, indexing, and querying geospatial data."
17. "Temporal Extensions" (https://wiki.postgresql.org/wiki/Temporal_Extensions). *PostgreSQL Wiki*. December 18, 2023. Retrieved December 18, 2023. "Postgres can be extended to become a Temporal Database. Such databases track the history of database content over time, automatically retaining said history and allowing it to be altered and queried."
18. "Orafce - Oracle's compatibility functions and packages" (<https://github.com/orafce/orafce>). *GitHub.com*. December 17, 2023. Retrieved December 18, 2023. "Functions and operators that emulate a subset of functions and packages from the Oracle RDBMS."
19. "pg_dbms_job" (https://github.com/MigOpsRepos/pg_dbms_job#readme). *GitHub.com*. November 8, 2023. Retrieved December 18, 2023. "PostgreSQL extension to schedules and manages jobs in a job queue similar to Oracle DBMS_JOB package."
20. "WiltonDB" (<https://wiltondb.com/>). *WiltonDB*. 2023. Retrieved December 18, 2023. "WiltonDB [is] packaged for Windows. It strives to be usable as a drop-in replacement to Microsoft SQL Server."
21. "Babelfish for PostgreSQL" (<https://babelfishpg.org/>). *babelfishpg.org*. Retrieved December 18, 2023. "Babelfish for PostgreSQL ... provides the capability for PostgreSQL to understand queries from applications written for Microsoft SQL Server."
22. "PostgreSQL Clients" (https://wiki.postgresql.org/wiki/PostgreSQL_Clients). *wiki.postgresql.org*. October 18, 2023. Retrieved December 18, 2023. "This page is a partial list of interactive SQL clients (GUI or otherwise) ... that you can type SQL in to and get results from them."
23. "Design Tools" (https://wiki.postgresql.org/wiki/Design_Tools). *wiki.postgresql.org*. October 23, 2023. Retrieved December 18, 2023. "Tools to help with designing a schema, via creating Entity-Relationship diagrams and similar. Most are GUI."
24. "Community Guide to PostgreSQL GUI Tools" (https://wiki.postgresql.org/wiki/Community_Guide_to_PostgreSQL_GUI_Tools). *wiki.postgresql.org*. December 1, 2023. Retrieved December 18, 2023. "This page is a list of miscellaneous utilities that work with Postgres (ex: data loaders, comparators etc.)."

25. "Replication, Clustering, and Connection Pooling" ([https://wiki.postgresql.org/wiki/Replication, Clustering, and Connection Pooling](https://wiki.postgresql.org/wiki/Replication,_Clustering,_and_Connection_Pooling)). *wiki.postgresql.org*. July 13, 2020. Retrieved December 18, 2023. "There are many approaches available to scale PostgreSQL beyond running on a single server. ... There is no one-size fits all..."
26. This is recognized by the liberal permission to use the PostgreSQL name, as approved (for fair use, when **not** confusing people about a legal relationship with the actual PostgreSQL project) when used in support of PostgreSQL, subject to the PostgreSQL Trademark Policy: "Trademark Policy" (<https://www.postgresql.org/about/policies/trademarks/>). *PostgreSQL.org*. December 8, 2020. Retrieved December 17, 2023. "We will try to work with you to permit uses [of the PostgreSQL name] that support the PostgreSQL project and our Community."
27. Stonebraker, M.; Rowe, L. A. (May 1986). *The design of POSTGRES* (<http://db.cs.berkeley.edu/papers/ERL-M85-95.pdf>) (PDF). Proc. 1986 ACM SIGMOD Conference on Management of Data. Washington, DC. Retrieved December 17, 2011.
28. "PostgreSQL: History" (<https://web.archive.org/web/20170326020245/https://www.postgresql.org/about/history/>). PostgreSQL Global Development Group. Archived from the original (<https://www.postgresql.org/about/history/>) on March 26, 2017. Retrieved August 27, 2016.
29. "Project name – statement from the core team" (<http://archives.postgresql.org/pgsql-advocacy/2007-11/msg00109.php>). *archives.postgresql.org*. November 16, 2007. Retrieved November 16, 2007.
30. "Michael Stonebraker – A.M. Turing Award Winner" (https://amturing.acm.org/award_winners/stonebraker_1172121.cfm). *amturing.acm.org*. Retrieved March 20, 2018. "Techniques pioneered in Postgres were widely implemented [...] Stonebraker is the only Turing award winner to have engaged in serial entrepreneurship on anything like this scale, giving him a distinctive perspective on the academic world."
31. Stonebraker, M.; Rowe, L. A. *The POSTGRES data model* (<http://db.cs.berkeley.edu/papers/ERL-M87-13.pdf>) (PDF). Proceedings of the 13th International Conference on Very Large Data Bases. Brighton, England: Morgan Kaufmann Publishers. pp. 83–96. ISBN 0-934613-46-X.
32. Pavel Stehule (June 9, 2012). "Historie projektu PostgreSQL" (http://postgres.cz/wiki/Historie_projektu_PostgreSQL) (in Czech).
33. A Brief History of PostgreSQL "Version 3 appeared in 1991 and added support for multiple storage managers, an improved query executor, and a rewritten rule system." (<https://www.postgresql.org/docs/9.3/history.html>). *postgresql.org*. *The PostgreSQL Global Development Group*, Retrieved on March 18, 2020.
34. "University POSTGRES, Version 4.2" (<http://db.cs.berkeley.edu/postgres.html>). July 26, 1999.
35. Page, Dave (April 7, 2015). "Re: 20th anniversary of PostgreSQL ?" (https://www.postgresql.org/message-id/CA+OCxozS_cuaLw=nfS=GdJZmS7ygjhdTzbqVt17wPLfCotFY4g@mail.gmail.com). *pgsql-advocacy* (Mailing list). Retrieved April 9, 2015.
36. Group, PostgreSQL Global Development (September 25, 2025). "PostgreSQL 18 Released!" (<https://www.postgresql.org/about/news/postgresql-18-released-3142/>). *PostgreSQL News*. Retrieved December 6, 2025. {{cite web}}: |last= has generic name (help)
37. Dan R. K. Ports; Kevin Grittnr (2012). "Serializable Snapshot Isolation in PostgreSQL" (<http://drkp.net/drkp/papers/ssi-vldb12.pdf>) (PDF). *Proceedings of the VLDB Endowment*. **5** (12): 1850–1861. arXiv:1208.4179 (<https://arxiv.org/abs/1208.4179>). Bibcode:2012arXiv1208.4179P (<https://ui.adsabs.harvard.edu/abs/2012arXiv1208.4179P>). doi:10.14778/2367502.2367523 (<https://doi.org/10.14778%2F2367502.2367523>). S2CID 16006111 (<https://api.semanticscholar.org/CorpusID:16006111>).
38. Bohan Zhang; Andy Pavlo (2023). "The part of PostgreSQL we hate the most" (<https://ottertune.com/blog/the-part-of-postgresql-we-hate-the-most>). *OtterTune* (blog).

39. *PostgreSQL 9.1 with synchronous replication* (<http://www.h-online.com/open/news/item/PostgreSQL-9-1-with-synchronous-replication-1341228.html>) (news), H Online
40. "26.4. Hot Standby" (<https://www.postgresql.org/docs/17/hot-standby.html>). *PostgreSQL Documentation*. May 8, 2025. Retrieved July 22, 2025.
41. Hood, Stu (June 30, 2025). "We Made Postgres Writes Faster, but it Broke Replication" (https://www.paradedb.com/blog/lsm_trees_in_postgres). *ParadeDB*. Retrieved July 22, 2025.
42. "Postgres-XC project page" (<https://web.archive.org/web/20120701122448/http://postgres-xc.sourceforge.net/>) (website). Postgres-XC. Archived from the original (<https://postgres-xc.sourceforge.net/>) on July 1, 2012.
43. "Postgres-R: a database replication system for PostgreSQL" (<https://web.archive.org/web/20100329215559/http://www.postgres-r.org/>). Postgres Global Development Group. Archived from the original (<http://www.postgres-r.org/>) on March 29, 2010. Retrieved August 27, 2016.
44. "Postgres-BDR" (<http://2ndquadrant.com/en/resources/bdr/>). 2ndQuadrant Ltd. Retrieved August 27, 2016.
45. Marit Fischer (November 10, 2007). "Backcountry.com finally gives something back to the open source community" (<https://web.archive.org/web/20101226124550/http://www.backcountrycorp.com/corporate/section/3/press/a511/Backcountry-finally-gives-something-back-to-the-open-source-community.html>) (Press release). Backcountry.com. Archived from the original (<http://www.backcountrycorp.com/corporate/section/3/press/a511/Backcountry-finally-gives-something-back-to-the-open-source-community.html>) on December 26, 2010.
46. Bartunov, O; Sigaev, T (May 2011). *SP-GiST – a new indexing framework for PostgreSQL* (http://www.pgcon.org/2011/schedule/attachments/197_pgcon-2011.pdf) (PDF). PGCon 2011. Ottawa, Canada. Retrieved January 31, 2016.
47. Bartunov, O; Sigaev, T (May 2010). *K-nearest neighbour search for PostgreSQL* (http://www.pgcon.org/2010/schedule/attachments/168_pgcon-2010-1.pdf) (PDF). PGCon 2010. Ottawa, Canada. Retrieved January 31, 2016.
48. "PostgreSQL, the NoSQL Database | Linux Journal" (<https://www.linuxjournal.com/content/postgresql-nosql-database>). *www.linuxjournal.com*.
49. Geoghegan, Peter (March 23, 2014). "What I think of jsonb" (<http://pgeoghegan.blogspot.com/2014/03/what-i-think-of-jsonb.html>).
50. Obe, Regina; Hsu, Leo S. (2012). "10: Replication and External Data". *PostgreSQL: Up and Running* (<https://books.google.com/books?id=Q8jklZkMTPcC>) (1 ed.). Sebastopol, CA: O'Reilly Media, Inc. p. 129. ISBN 978-1-4493-2633-3. Retrieved October 17, 2016. "Foreign Data Wrappers (FDW) [...] are mechanisms of querying external datasources. PostgreSQL 9.1 introduced this SQL/MED standards compliant feature."
51. "Frontend/Backend Protocol" (<https://www.postgresql.org/docs/current/protocol.html>). *postgresql.org*. November 9, 2023. Retrieved December 17, 2023. "This document describes version 3.0 of the protocol, implemented in PostgreSQL 7.4 and later."
52. "libpq" (<https://www.postgresql.org/docs/16/libpq.html>). *postgresql.org*. November 9, 2023. Retrieved December 17, 2023.
53. "Embedded SQL in C" (<https://www.postgresql.org/docs/16/ecpg.html>). *postgresql.org*. November 9, 2023. Retrieved December 17, 2023.
54. "Client Interfaces" (<https://www.postgresql.org/docs/16/client-interfaces.html>). *postgresql.org*. November 9, 2023. Retrieved December 17, 2023.
55. "libpqxx" (<https://pqxx.org/development/libpqxx/>). Retrieved April 4, 2020.
56. "PostgreSQL JDBC Driver" (<https://jdbc.postgresql.org/>). Retrieved April 4, 2020.
57. "[ANN] PostgresORM.jl: Object Relational Mapping for PostgreSQL" (<https://discourse.julialang.org/t/ann-postgresorm-jl-object-relational-mapping-for-postgresql/63847>). *JuliaLang*. June 30, 2021. Retrieved August 26, 2021.
58. "GitHub - invenia/LibPQ.jl: A Julia wrapper for libpq" (<https://github.com/invenia/LibPQ.jl>). *GitHub*. Retrieved August 26, 2021.

59. "PL/Julia extension (minimal)" (<https://discourse.julialang.org/t/pl-julia-extension-minimal/34232/2>). *JuliaLang*. March 8, 2020. Retrieved August 26, 2021.
60. "PostgreSQL + Python | Psycopg" (<https://web.archive.org/web/20121101032433/http://initd.org/psycopg/>). *initd.org*. Archived from the original (<http://initd.org/psycopg/>) on November 1, 2012. Retrieved January 21, 2015.
61. "node-postgres" (<https://node-postgres.com/>). Retrieved April 4, 2020.
62. "SQL database drivers" (<https://github.com/golang/go/wiki/SQLDrivers#drivers>). *Go wiki*. *golang.org*. Retrieved June 22, 2015.
63. "Rust-Postgres" (<https://crates.io/crates/postgres>). Retrieved April 4, 2020.
64. "Server Programming" (<https://www.postgresql.org/docs/current/server-programming.html>). *PostgreSQL documentation*. Retrieved May 19, 2019.
65. "DO" (<https://www.postgresql.org/docs/current/sql-do.html>). *PostgreSQL documentation*. Retrieved May 19, 2019.
66. "PL/Python - Python Procedural Language" (<https://www.postgresql.org/docs/current/plpython.html>). *PostgreSQL documentation*. Retrieved October 23, 2022.
67. "Procedural Languages" (<https://www.postgresql.org/docs/current/static/external-pl.html>). *postgresql.org*. March 31, 2016. Retrieved April 7, 2016.
68. "postgres-plr/plr" (<https://github.com/postgres-plr/plr>). June 17, 2021 – via GitHub.
69. Chartier, Colin (November 8, 2019). "System design hack: Postgres is a great pub/sub & job server" (<https://layerci.com/blog/postgres-is-the-answer/>). *LayerCI blog*. Retrieved November 24, 2019.
70. "Release 9.5" (<https://www.postgresql.org/docs/9.5/release-9-5.html>). *postgresql.org*. February 11, 2021.
71. Ringer, Craig (April 13, 2016). "What is SKIP LOCKED for in PostgreSQL 9.5?" (<https://www.2ndquadrant.com/en/blog/what-is-select-skip-locked-for-in-postgresql-9-5/>). *2nd Quadrant*. Retrieved November 24, 2019.
72. "Add a materialized view relations" (<https://www.postgresql.org/message-id/E1UCJDN-00042x-0w@gemulon.postgresql.org>). March 4, 2013. Retrieved March 4, 2013.
73. "Support automatically-updatable views" (<http://archives.postgresql.org/pgsql-committers/2012-12/msg00154.php>). December 8, 2012. Retrieved December 8, 2012.
74. "Add CREATE RECURSIVE VIEW syntax" (<https://www.postgresql.org/message-id/E1U17NB-0006c6-DX@gemulon.postgresql.org>). February 1, 2013. Retrieved February 28, 2013.
75. Momjian, Bruce (2001). "Subqueries" (https://web.archive.org/web/20100809013228/http://www.postgresql.org/files/documentation/books/aw_pgsql/15467.html). *PostgreSQL: Introduction and Concepts* (https://www.postgresql.org/files/documentation/books/aw_pgsql/15467.html). Addison-Wesley. ISBN 0-201-70331-9. Archived from the original (https://www.postgresql.org/files/documentation/books/aw_pgsql/node81.html) on August 9, 2010. Retrieved September 25, 2010.
76. Bernier, Robert (February 2, 2006). "Using Regular Expressions in PostgreSQL" (http://www.oreillynet.com/pub/a/databases/2006/02/02/postgresq_regexes.html). O'Reilly Media. Retrieved September 25, 2010.
77. "A few short notes about PostgreSQL and POODLE" (<http://blog.hagander.net/archives/222-A-few-short-notes-about-PostgreSQL-and-POODLE.html>). *hagander.net*.
78. Berkus, Josh (June 2, 2016). "PostgreSQL 9.6 Beta and PGCon 2016" (<https://lwn.net/Articles/689387/>). *LWN.net*.
79. "FAQ – PostgreSQL wiki" (https://wiki.postgresql.org/wiki/FAQ#How_does_PostgreSQL_use_CPU_resources.3F). *wiki.postgresql.org*. Retrieved April 13, 2017.
80. "SEPostgreSQL Documentation – PostgreSQL wiki" (https://wiki.postgresql.org/wiki/SEPostgreSQL_Documentation). *wiki.postgresql.org*.

81. "NB SQL 9.3 - SELinux Wiki" (https://selinuxproject.org/page/NB_SQL_9.3). *selinuxproject.org*.
82. "PostgreSQL 10 Documentation: Appendix E. Release Notes" (<https://www.postgresql.org/docs/10/release-10.html#id-1.11.6.16.3>). August 12, 2021.
83. "PostgreSQL: About" (<https://www.postgresql.org/about/>). *www.postgresql.org*. Retrieved October 14, 2024.
84. "Case sensitivity of identifiers" (<https://www.postgresql.org/docs/current/static/sql-syntax-lexical.html#SQL-SYNTAX-IDENTIFIERS>). PostgreSQL Global Development Group. November 11, 2021.
85. Berkus, Josh (July 6, 2007). "PostgreSQL publishes first real benchmark" (<https://web.archive.org/web/20070712092901/http://blogs.ittoolbox.com/database/soup/archives/postgresql-publishes-first-real-benchmark-17470>). Archived from the original (<http://blogs.ittoolbox.com/database/soup/archives/postgresql-publishes-first-real-benchmark-17470>) on July 12, 2007. Retrieved July 10, 2007.
86. Vilmos, György (September 29, 2009). "PostgreSQL history" (http://suckit.blog.hu/2009/09/29/postgresql_history). Retrieved August 28, 2010.
87. "SPECjAppServer2004 Result" (<http://www.spec.org/jAppServer2004/results/res2007q3/jAppServer2004-20070606-00065.html>). SPEC. July 6, 2007. Retrieved July 10, 2007.
88. "SPECjAppServer2004 Result" (<http://www.spec.org/jAppServer2004/results/res2007q3/jAppServer2004-20070703-00073.html>). SPEC. July 4, 2007. Retrieved September 1, 2007.
89. "Managing Kernel Resources" (<https://www.postgresql.org/docs/current/static/kernel-resources.html>). *PostgreSQL Manual*. PostgreSQL.org. Retrieved November 12, 2011.
90. Greg Smith (October 15, 2010). *PostgreSQL 9.0 High Performance* (<http://www.packtpub.com/postgresql-90-high-performance/book>). Packt Publishing. ISBN 978-1-84951-030-1.
91. Robert Haas (April 3, 2012). "Did I Say 32 Cores? How about 64?" (<http://rhaas.blogspot.com/2012/04/did-i-say-32-cores-how-about-64.html>). Retrieved April 8, 2012.
92. Khushi, Matloob (June 2015). "Benchmarking database performance for genomic data". *J Cell Biochem*. **116** (6): 877–83. arXiv:2008.06835 (<https://arxiv.org/abs/2008.06835>). doi:10.1002/jcb.25049 (<https://doi.org/10.1002%2Fjcb.25049>). PMID 25560631 (<https://pubmed.ncbi.nlm.nih.gov/25560631>). S2CID 27458866 (<https://api.semanticscholar.org/CorpusID:27458866>).
93. "Mac OS X packages" (<https://www.postgresql.org/download/macosx/>). The PostgreSQL Global Development Group. Retrieved August 27, 2016.
94. "PostgreSQL: Windows installers" (<https://www.postgresql.org/download/windows/>). *www.postgresql.org*. Retrieved August 26, 2021.
95. "postgresql-client-10.5p1 – PostgreSQL RDBMS (client)" (<http://ports.su/databases/postgresql-main>). *OpenBSD ports*. October 4, 2018. Retrieved October 10, 2018.
96. "Installing and Configuring PostgreSQL - Oracle Solaris Cluster Data Service for PostgreSQL Guide" (<https://docs.oracle.com/cd/E19680-01/html/821-1534/ciajejfa.html>). *docs.oracle.com*. Retrieved February 4, 2023.
97. "AArch64 planning BoF at DebConf" (<http://lists.debian.org/debian-devel/2012/07/msg00536.html>). *debian.org*.
98. Souza, Rubens (June 17, 2015). "Step 5 (update): Installing PostgreSQL on my Raspberry Pi 1 and 2" (<https://web.archive.org/web/20160920031816/http://raspberrypg.org/2015/06/step-5-update-installing-postgresql-on-my-raspberry-pi-1-and-2/>). Archived from the original (<http://raspberrypg.org/2015/06/step-5-update-installing-postgresql-on-my-raspberry-pi-1-and-2/>) on September 20, 2016. Retrieved August 27, 2016.
99. "Supported Platforms" (<https://www.postgresql.org/docs/current/static/supported-platforms.html>). PostgreSQL Global Development Group. Retrieved April 6, 2012.
100. "Available lexers — Pygments" (<https://pygments.org/docs/lexers/#pygments.lexers.sql.PostgresConsoleLexer>).

101. "pgAdmin: PostgreSQL administration and management tools" (<http://www.pgadmin.org/>). *website*. Retrieved November 12, 2011.
102. "Debian -- Details of package pgadmin3 in jessie" (<https://packages.debian.org/jessie/pgadmin3>). Retrieved March 10, 2017.
103. "pgAdmin Development Team" (<http://www.pgadmin.org/development/team.php>). *pgadmin.org*. Retrieved June 22, 2015.
104. Dave, Page (December 7, 2014). "The story of pgAdmin" (<http://pgsnake.blogspot.co.uk/2014/12/the-story-of-pgadmin.html>). *Dave's Postgres Blog*. pgsnake.blogspot.co.uk. Retrieved December 7, 2014.
105. "pgAdmin 4 README" (<https://github.com/postgres/pgadmin4/blob/master/README>). *GitHub*. Retrieved August 15, 2018.
106. phpPgAdmin Project (April 25, 2008). "About phpPgAdmin" (<https://phppgadmin.sourceforge.net/?page=about>). Retrieved April 25, 2008.
107. PostgreSQL Studio (October 9, 2013). "About PostgreSQL Studio" (<https://web.archive.org/web/20131007084849/http://www.postgresqlstudio.org/about/>). Archived from the original (<http://www.postgresqlstudio.org/about/>) on October 7, 2013. Retrieved October 9, 2013.
108. "TeamPostgreSQL website" (<http://www.teampostgresql.com>). October 3, 2013. Retrieved October 3, 2013.
109. oooforum.org (January 10, 2010). "Back Ends for OpenOffice" (<https://web.archive.org/web/20110928093709/http://www.oooforum.org/forum/viewtopic.phtml?p=356180>). Archived from the original (<http://www.oooforum.org/forum/viewtopic.phtml?p=356180>) on September 28, 2011. Retrieved January 5, 2011.
110. libreoffice.org (October 14, 2012). "Base features" (<https://web.archive.org/web/20120107063659/http://www.libreoffice.org/features/base/>). Archived from the original (<http://www.libreoffice.org/features/base/>) on January 7, 2012. Retrieved October 14, 2012.
111. Greg Smith; Robert Treat & Christopher Browne. "Tuning your PostgreSQL server" (https://wiki.postgresql.org/wiki/Tuning_Your_PostgreSQL_Server). *Wiki*. PostgreSQL.org. Retrieved November 12, 2011.
112. "pgDevOps" (<https://web.archive.org/web/20170401220832/http://www1.bigsql.org/pgdevops/>). *BigSQL.org*. Archived from the original (<https://www.bigsql.org/pgdevops/>) on April 1, 2017. Retrieved May 4, 2017.
113. "pgbackrest/pgbackrest" (<https://github.com/pgbackrest/pgbackrest>). *GitHub*. November 21, 2021.
114. "pgaudit/pgaudit" (<https://github.com/pgaudit/pgaudit>). *GitHub*. November 21, 2021.
115. "wal-e/wal-e" (<https://github.com/wal-e/wal-e>). June 24, 2021 – via GitHub.
116. Dugin, Rostislav (November 28, 2025), *RostislavDugin/postgresus* (<https://github.com/RostislavDugin/postgresus>), retrieved November 28, 2025
117. Claire Giordano (October 31, 2019). "Architecting petabyte-scale analytics by scaling out Postgres on Azure with the Citus extension" (<https://techcommunity.microsoft.com/t5/azure-database-for-postgresql/architecting-petabyte-scale-analytics-by-scaling-out-postgres-on-ba-p/969685>). *Blog*. Microsoft Tech Community.
118. Emmanuel Cecchet (May 21, 2009). *Building PetaByte Warehouses with Unmodified PostgreSQL* (http://www.pgcon.org/2009/schedule/attachments/135_PGCon%202009%20-%20Aster%20v6.pdf) (PDF). PGCon 2009. Retrieved November 12, 2011.
119. "MySpace.com scales analytics for all their friends" (http://www.asterdata.com/resources/assets/cs_Aster_Data_4.0_MySpace.pdf) (PDF). case study. Aster Data. June 15, 2010. Archived (https://web.archive.org/web/20101114141918/http://asterdata.com/resources/assets/cs_Aster_Data_4.0_MySpace.pdf) (PDF) from the original on November 14, 2010. Retrieved November 12, 2011.
120. "Last Weekend's Outage" (<http://www.geni.com/blog/last-weekends-outage-368211.html>). *Blog*. Geni. August 1, 2011.

121. "Database" (<https://wiki.openstreetmap.org/wiki/Database>). Wiki. OpenStreetMap.
122. *PostgreSQL affiliates .ORG domain* (<https://www.computerworld.com.au/index.php?id=760310963>), Australia: Computer World, August 24, 2023
123. W. Jason Gilmore; R.H. Treat (2006). *Beginning PHP and PostgreSQL 8: From Novice to Professional* (<https://books.google.com/books?id=BiRC4JtQzFIC&pg=PA577>). Apress. ISBN 978-1-43020-136-6. Retrieved August 30, 2017.
124. *Sony Online opts for open-source database over Oracle* (<https://www.computerworld.com/databasetopics/data/software/story/0,10801,109722,00.html>), Computer World
125. "A Web Commerce Group Case Study on PostgreSQL" (<https://www.postgresql.org/files/about/casestudies/wcgcasestudyonpostgresqlv1.2.pdf>) (PDF) (1.2 ed.). PostgreSQL.
126. "Architecture Overview" (<https://github.com/reddit/reddit/wiki/Architecture-Overview#reddit-the-software>). *Reddit software wiki*. Reddit. March 27, 2014. Retrieved November 25, 2014.
127. Pihlak, Martin. "PostgreSQL @Skype" (<https://wiki.postgresql.org/images/a/a9/Postgresql-at-skype.pdf>) (PDF). *wiki.postgresql.org*. Retrieved January 16, 2019.
128. "How Much Are You Paying For Your Database?" (https://web.archive.org/web/20090307032257/http://blogs.sun.com/marchamilton/entry/how_much_are_you_paying). Sun Microsystems blog. 2007. Archived from the original (http://blogs.sun.com/marchamilton/entry/how_much_are_you_paying) on March 7, 2009. Retrieved December 14, 2007.
129. "Database – MusicBrainz" (<http://musicbrainz.org/doc/Database>). MusicBrainz Wiki. Retrieved February 5, 2011.
130. Duncavage, Daniel P (July 13, 2010). "NASA needs Postgres-Nagios help" (<http://archives.postgresql.org/pgsql-general/2010-07/msg00394.php>).
131. Roy, Gavin M (2010). "PostgreSQL at myYearbook.com" (https://web.archive.org/web/20110727183016/https://www.postgresqlconference.org/2010/east/talks/postgresql_at_myyearbook.com) (talk). USA East: PostgreSQL Conference. Archived from the original (https://www.postgresqlconference.org/2010/east/talks/postgresql_at_myyearbook.com) on July 27, 2011.
132. "Keeping Instagram up with over a million new users in twelve hours" (<https://instagram-engineering.tumblr.com/post/20541814340/keeping-instagram-up-with-over-a-million-new-users-in#replicationread-slaves>). Instagram-engineering.tumblr.com. May 17, 2011. Retrieved July 7, 2012.
133. "Postgres at Disqus" (<https://speakerdeck.com/mikeclarke/pgcon-2013-keynote-postgres-at-disqus>). Retrieved May 24, 2013.
134. Kelly, Matthew (March 27, 2015). *At the Heart of a Giant: Postgres at TripAdvisor* (<https://web.archive.org/web/20150723181100/http://www.pgconf.us/2015/event/95/>). PGConf US 2015. Archived from the original (<http://www.pgconf.us/2015/event/95/>) on July 23, 2015. Retrieved July 23, 2015. (Presentation video (<https://www.youtube.com/watch?v=YquXmwZNnfg>))
135. "Yandex.Mail's successful migration from Oracle to Postgres [pdf]" (<https://news.ycombinator.com/item?id=12489055>). *Hacker News: news.ycombinator.com*. Retrieved September 28, 2016.
136. S. Riggs; G. Ciolli; H. Krosing; G. Bartolini (2015). *PostgreSQL 9 Administration Cookbook - Second Edition* (<https://books.google.com/books?id=rYrwCAAAQBAJ&pg=PA3>). Packt. ISBN 978-1-84951-906-9. Retrieved September 5, 2017.
137. "Met Office swaps Oracle for PostgreSQL" (<https://www.computerweekly.com/ezone/Computer-Weekly/The-Met-Office-turns-to-open-source/Met-Office-swaps-Oracle-for-PostgreSQL>). *computerweekly.com*. June 17, 2014. Retrieved September 5, 2017.
138. "Open Source Software" (<https://flightaware.com/about/code/>). *FlightAware*. Retrieved November 22, 2017.
139. "Ansible at Grofers (Part 2) — Managing PostgreSQL" (<https://lambda.grofers.com/ansible-at-grofers-part-2-managing-postgresql-c4069ce5855b>). *Lambda - The Grofers Engineering Blog*. February 28, 2017. Retrieved September 5, 2018.

140. McMahon, Philip; Chiorean, Maria-Livia; Coleman, Susie; Askoolum, Akash (November 30, 2018). "Digital Blog: Bye bye Mongo, Hello Postgres" (<https://www.theguardian.com/info/2018/nov/30/bye-bye-mongo-hello-postgres>). *The Guardian*. ISSN 0261-3077 (<https://search.worldcat.org/issn/0261-3077>).
141. "Elevated Errors on API and ChatGPT" (<https://status.openai.com/incidents/n254wyd7nml7>). November 21, 2023. Retrieved December 2, 2023.
142. "Microsoft FY25 Third Quarter Earnings Conference Call" (<https://view.officeapps.live.com/o/p/view.aspx?src=https://microsoft.com/en-us/investor/earnings/FY-2025-Q3/Document/DownloadDocument/63/TranscriptQandAFY25q3.docx>). May 5, 2025.
143. Alex Williams (April 1, 2013). "Heroku Forces Customer Upgrade To Fix Critical PostgreSQL Security Hole" (<https://techcrunch.com/2013/04/01/heroku-forces-customer-upgrade-to-fix-critical-postgresql-security-hole/>). *TechCrunch*.
144. Barb Darrow (November 11, 2013). "Heroku gussies up Postgres with database roll-back and proactive alerts" (<https://web.archive.org/web/20131112182809/http://gigaom.com/2013/11/11/heroku-gussies-up-postgres-with-database-roll-back-and-proactive-alerts/>). GigaOM. Archived from the original (<http://gigaom.com/2013/11/11/heroku-gussies-up-postgres-with-database-roll-back-and-proactive-alerts/>) on November 12, 2013.
145. Craig Kerstiens (September 26, 2013). "WAL-E and Continuous Protection with Heroku Postgres" (https://blog.heroku.com/archives/2013/9/26/wal_e_and_continuous_protection_with_heroku_postgres). Heroku blog.
146. "EnterpriseDB Offers Up Postgres Plus Cloud Database" (<https://web.archive.org/web/20120130040758/http://www.techweekeurope.co.uk/news/enterprisedb-offers-up-postgres-plus-cloud-database-57030>). Techweekeurope.co.uk. January 27, 2012. Archived from the original on January 30, 2012. Retrieved July 7, 2012.
147. "Alibaba Cloud Expands Technical Partnership with EnterpriseDB" (<https://www.milestonepartners.com/alibaba-cloud-expands-technical-partnership-with-enterprisedb/>). *Milestone Partners*. September 26, 2018. Retrieved June 9, 2020.
148. O'Doherty, Paul; Asselin, Stephane (2014). "3: VMware Workspace Architecture". *VMware Horizon Suite: Building End-User Services* (<https://books.google.com/books?id=1mTYAwAAQBAJ>). VMware Press Technology. Upper Saddle River, NJ: VMware Press. p. 65. ISBN 978-0-13-347910-2. Retrieved September 19, 2016. "In addition to the open source version of PostgreSQL, VMware offers vFabric Postgres, or vPostgres. vPostgres is a PostgreSQL virtual appliance that has been tuned for virtual environments."
149. Al Sargent (May 15, 2012). "Introducing VMware vFabric Suite 5.1: Automated Deployment, New Components, and Open Source Support" (<https://blogs.vmware.com/vfabric/2012/05/announcing-vmware-vfabric-suite-51.html>). VMware blogs.
150. "VMware vFabric Suite EOA" (<https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/products/VMware-vFabric-Suite-EOA-FAQ.pdf>) (PDF). September 1, 2014. Retrieved December 17, 2023.
151. Jeff (November 14, 2013). "Amazon RDS for PostgreSQL – Now Available" (<http://aws.typepad.com/aws/2013/11/amazon-rds-for-postgresql-now-available.html>). Amazon Web Services Blog.
152. Alex Williams (November 14, 2013). "PostgreSQL Now Available On Amazon's Relational Database Service" (<https://techcrunch.com/2013/11/14/postgresql-now-available-on-amazon-relational-database-service/>). *TechCrunch*.
153. "Amazon Aurora Update – PostgreSQL Compatibility" (<https://aws.amazon.com/blogs/aws/amazon-aurora-update-postgresql-compatibility/>). *AWS Blog*. November 30, 2016. Retrieved December 1, 2016.
154. "Announcing Azure Database for PostgreSQL" (<https://azure.microsoft.com/en-us/blog/azure-database-for-mysql-public-preview/>). *Azure Blog*. May 10, 2017. Retrieved June 19, 2019.

155. "Aliyun PolarDB released major updates to support one-click migration of databases such as Oracle to the cloud" (<https://developpaper.com/aliyun-polaradb-released-major-updates-to-support-one-click-migration-of-databases-such-as-oracle-to-the-cloud/>). *Develop Paper*. July 6, 2019.
156. "Asynchronous Master-Slave Replication of PostgreSQL Databases in One Click" (<https://dzone.com/articles/asynchronous-master-slave-replication-of-postgresql>). *DZone*. Retrieved May 26, 2017.
157. "IBM Cloud Hyper Protect DBaaS for PostgreSQL documentation" (<https://cloud.ibm.com/docs/hyper-protect-dbaas-for-postgresql?topic=hyper-protect-dbaas-for-postgresql-gettingstarted>). *cloud.ibm.com*. Retrieved June 24, 2020.
158. "Crunchy Data Continues PostgreSQL Support with the Release of Crunchy Bridge" (<https://www.dbta.com/Editorial/News-Flashes/Crunchy-Data-Continues-PostgreSQL-Support-with-the-Release-of-Crunchy-Bridge-142951>). September 18, 2020.
159. "What's New in PostgreSQL 14?" (<https://render.com/blog/whats-new-in-pg14>). *render.com*. Retrieved May 26, 2026.
160. "SELECT 'Hello, World' Serverless Postgres built for the cloud" (<https://neon.tech/blog/hello-world>). June 15, 2022.
161. "AlloyDB for PostgreSQL fast high-availability cloud database - Google Cloud Blog" (<https://cloud.google.com/blog/products/databases/announcing-the-general-availability-of-alloydb-for-postgresql>). March 6, 2025.
162. "Introducing Nile, Serverless Postgres for modern SaaS" (<https://www.thenile.dev/blog/app/blog/introducing-nile>). October 25, 2023.
163. "Announcing the general availability of AlloyDB Omni" (<https://cloud.google.com/blog/products/databases/announcing-the-general-availability-of-alloydb-omni>). *Google Cloud Blog*. April 10, 2024.
164. "PlanetScale for Postgres is now GA — PlanetScale" (<https://planetscale.com/blog/planetscale-for-postgres-is-generally-available>). *planetscale.com*. September 22, 2025. Retrieved April 28, 2026.
165. "Versioning policy" (<https://www.postgresql.org/support/versioning/>). PostgreSQL Global Development Group. Retrieved October 4, 2018.
166. Vaas, Lisa (December 2, 2002). "Databases Target Enterprises" (<https://www.eweek.com/c/a/Database/Databases-Target-Enterprises>). *eWeek*. Retrieved October 29, 2016.
167. Krill, Paul (November 20, 2003). "PostgreSQL boosts open source database" (<https://www.infoworld.com/article/2670451/database/postgresql-boosts-open-source-database.html>). *InfoWorld*. Retrieved October 21, 2016.
168. Krill, Paul (January 19, 2005). "PostgreSQL open source database boasts Windows boost" (<https://www.infoworld.com/article/2668622/operating-systems/postgresql-open-source-database-boasts-windows-boost.html>). *InfoWorld*. Retrieved November 2, 2016.
169. Weiss, Todd R. (December 5, 2006). "Version 8.2 of open-source PostgreSQL DB released" (<https://www.computerworld.com/article/2548483>). *Computerworld*. Retrieved October 17, 2016.
170. Gilbertson, Scott (February 5, 2008). "PostgreSQL 8.3: Open Source Database Promises Blazing Speed" (https://www.wired.com/2008/02/postgresql_8dot3_open_source_database_promises_blazing_speed/). *Wired*. Retrieved October 17, 2016.
171. Huber, Mathias (July 2, 2009). "PostgreSQL 8.4 Proves Feature-Rich" ([https://www.linux-magazine.com/Online/News/PostgreSQL-8.4-Proves-Feature-Rich/\(language\)/eng-US](https://www.linux-magazine.com/Online/News/PostgreSQL-8.4-Proves-Feature-Rich/(language)/eng-US)). *Linux Magazine*. Retrieved October 17, 2016.
172. Brockmeier, Joe (September 30, 2010). "Five Enterprise Features in PostgreSQL 9" (<https://www.linux.com/news/five-enterprise-features-postgresql-9>). *Linux.com*. Linux Foundation. Retrieved February 6, 2017.

173. Timothy Prickett Morgan (September 12, 2011). "PostgreSQL revs to 9.1, aims for enterprise" (https://www.theregister.co.uk/2011/09/12/postgresql_9_1_cloud_server/). *The Register*. Retrieved February 6, 2017.
174. "PostgreSQL: PostgreSQL 9.2 released" (<https://www.postgresql.org/about/news/1415/>). *www.postgresql.org*. September 10, 2012.
175. "Reintroducing Hstore for PostgreSQL" (<https://www.infoq.com/news/2013/11/Nested-Hstore>). *InfoQ*.
176. Richard, Chirgwin (January 7, 2016). "Say oops, UPSERT your head: PostgreSQL version 9.5 has landed" (https://www.theregister.co.uk/2016/01/07/postgresql_95_lands/). *The Register*. Retrieved October 17, 2016.
177. "PostgreSQL: Documentation: 10: Chapter 31. Logical Replication" (<https://www.postgresql.org/docs/10/logical-replication.html>). *www.postgresql.org*. August 12, 2021.
178. "PostgreSQL 11 Released" (<https://www.postgresql.org/about/news/1894/>). October 18, 2018. Retrieved October 18, 2018.
179. "PostgreSQL Release Notes" (<https://www.postgresql.org/docs/11/static/release-11.html>). Retrieved October 18, 2018.
180. "PostgreSQL: PostgreSQL 12 Released!" (<https://www.postgresql.org/about/news/1976/>). *Postgresql News*. October 3, 2019.
181. "PostgreSQL 13 Release Notes" (<https://www.postgresql.org/docs/13/release-13.html>). *www.postgresql.org*. August 12, 2021.
182. "PostgreSQL 13 Released!" (<https://www.postgresql.org/about/news/postgresql-13-released-2077/>). *www.postgresql.org*. September 24, 2020.
183. "PostgreSQL 14 Release Notes" (<https://www.postgresql.org/docs/14/release-14.html>). *www.postgresql.org*. November 11, 2021.
184. "PostgreSQL 14 Released!" (<https://www.postgresql.org/about/news/postgresql-14-released-2318/>). *www.postgresql.org*. September 30, 2021.
185. "PostgreSQL 16 Released!" (<https://www.postgresql.org/about/news/postgresql-16-released-2715/>). September 14, 2023.
186. "PostgreSQL 17 Released!" (<https://www.postgresql.org/about/news/postgresql-17-released-2936/>). September 26, 2024.
187. "PostgreSQL 18 Released!" (<https://www.postgresql.org/about/news/postgresql-18-released-3142/>). September 25, 2025.
188. "Greenplum Database Documentation" (<https://scalegrid.io/blog/what-is-greenplum-database>). *docs.greenplum.org*. Retrieved August 10, 2025.
189. "About Timescale" (<https://www.timescale.com/about>). *Timescale*. Retrieved August 10, 2025. "TimescaleDB is implemented as an extension on PostgreSQL, which means that it runs within a PostgreSQL instance and that you interact with it in the same way that you interact with PostgreSQL."
190. "Amazon Aurora with PostgreSQL compatibility" (<https://docs.aws.amazon.com/AuroraRD/latest/AuroraUserGuide/AuroraPostgreSQL.html>). *Amazon Web Services, Inc.* Retrieved August 10, 2025.
191. "What is Neon?" (<https://neon.com/docs/get-started/why-neon>). *Neon*. Retrieved August 10, 2025. "Neon is a fully managed serverless PostgreSQL. It separates storage and compute and substitutes PostgreSQL's storage layer with a custom-built, multi-tenant storage engine designed for the cloud."
192. "AlloyDB overview" (<https://cloud.google.com/alloydb/docs/overview>). *Google Cloud*. Retrieved August 10, 2025. "AlloyDB for PostgreSQL is a fully managed, PostgreSQL-compatible database service that's designed for your most demanding workloads, including hybrid transactional and analytical processing... AlloyDB employs a disaggregated architecture. The compute and storage layers are separate and offer individual scaling."

193. "Introducing Database Traffic Control — PlanetScale" (<https://planetscale.com/blog/introducing-database-traffic-control>). *planetscale.com*. March 23, 2026. Retrieved April 28, 2026.

Further reading

- Obe, Regina; Hsu, Leo (July 8, 2012). *PostgreSQL: Up and Running* (<http://www.postgresql.com/store.php?asin=1449326331>). O'Reilly. ISBN 978-1-4493-2633-3.
- Krosing, Hannu; Roybal, Kirk (June 15, 2013). *PostgreSQL Server Programming* (<http://www.2ndquadrant.com/books/>) (second ed.). Packt Publishing. ISBN 978-1-84951-698-3.
- Riggs, Simon; Krosing, Hannu (October 27, 2010). *PostgreSQL 9 Administration Cookbook* (<http://www.2ndquadrant.com/books/>) (second ed.). Packt Publishing. ISBN 978-1-84951-028-8.
- Smith, Greg (October 15, 2010). *PostgreSQL 9 High Performance* (<http://www.2ndquadrant.com/books/>). Packt Publishing. ISBN 978-1-84951-030-1.
- Gilmore, W. Jason; Treat, Robert (February 27, 2006). *Beginning PHP and PostgreSQL 8: From Novice to Professional* (<https://web.archive.org/web/20090708113944/http://www.apress.com/book/view/1590595475>). Apress. p. 896. ISBN 1-59059-547-5. Archived from the original (<http://www.apress.com/book/view/1590595475>) on July 8, 2009. Retrieved April 28, 2009.
- Douglas, Korry (August 5, 2005). *PostgreSQL* (<http://www.informit.com/store/product.aspx?isbn=0672327562>) (second ed.). Sams. p. 1032. ISBN 0-672-32756-2.
- Matthew, Neil; Stones, Richard (April 6, 2005). *Beginning Databases with PostgreSQL* (<https://web.archive.org/web/20090409150911/http://www.apress.com/book/view/9781590594780>) (second ed.). Apress. p. 664. ISBN 1-59059-478-9. Archived from the original (<http://www.apress.com/book/view/9781590594780>) on April 9, 2009. Retrieved April 28, 2009.
- Worsley, John C.; Drake, Joshua D. (January 2002). *Practical PostgreSQL* (<https://archive.org/details/practicalpostgre00wors/page/636>). O'Reilly Media. pp. 636 (<https://archive.org/details/practicalpostgre00wors/page/636>). ISBN 1-56592-846-6.

External links

- Official website (<https://www.postgresql.org>) , and wiki (<https://wiki.postgresql.org/>)
- A Software Catalog (<https://www.postgresql.org/download/product-categories/>) of related projects and products
- The official Main Source Code Repository (for browsing) (<https://git.postgresql.org/gitweb/?p=postgresql.git>), and the Developer FAQ (https://wiki.postgresql.org/wiki/Developer_FAQ)
- The official Reference for PostgreSQL Documentation Authors (<https://www.postgresql.org/docs/current/docguide.html>)
- All official PostgreSQL Source Code Repositories (<https://git.postgresql.org/>)
- PostgreSQL (<https://github.com/postgres>) on GitHub

Retrieved from "<https://en.wikipedia.org/w/index.php?title=PostgreSQL&oldid=1365200359>"